Building an algorithm that can play Ultimate Tic-Tac-Toe

Suraj Gupta and Nehul Malhotra

Abstract

This paper aims to develop an optimal algorithm to play Ultimate Tic-Tac-Toe (UTTT) against human opponents. Unlike standard Tic-Tac-Toe, UTTT introduces an additional strategic layer by restricting moves based on the previous turn, significantly increasing complexity and requiring deeper planning. By testing different parameter values and comparing their performance, the paper identifies combinations that improve decision-making and consistency. The goal is to compare a heuristic algorithm to one that uses minimax as its fundamental decision-making method, while demonstrating how the heuristic algorithm succeeds in normal tic-tac-toe but fails in UTTT, in an environment where decision-making is heavily dependent on risk-taking for future gains.

A Heuristic Algorithm for Regular Tic-Tac-Toe

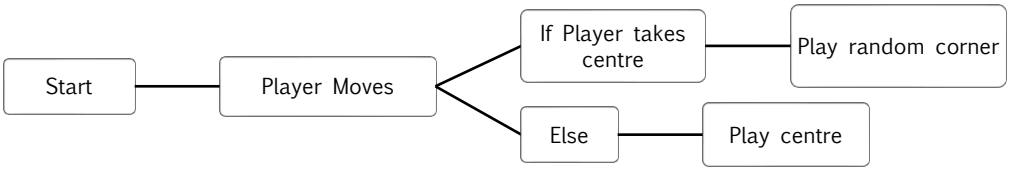
We began by developing an algorithm for standard Tic-Tac-Toe to better understand the decision structure required to implement an algorithm capable of playing Ultimate Tic-Tac-Toe.

The Algorithm

It is a handcrafted heuristic algorithm implemented in a loop that runs until the game is won, lost, or drawn. An important observation is that the algorithm isn't required for the first 3 moves of the game.

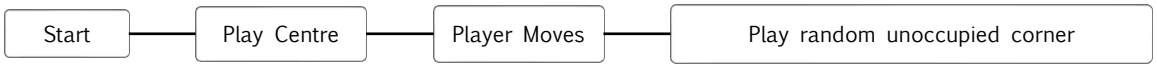
There are 2 situations for the first 3 moves:

- **Player-Bot-Player** if the player starts
- **Bot-Player-Bot** if the bot starts



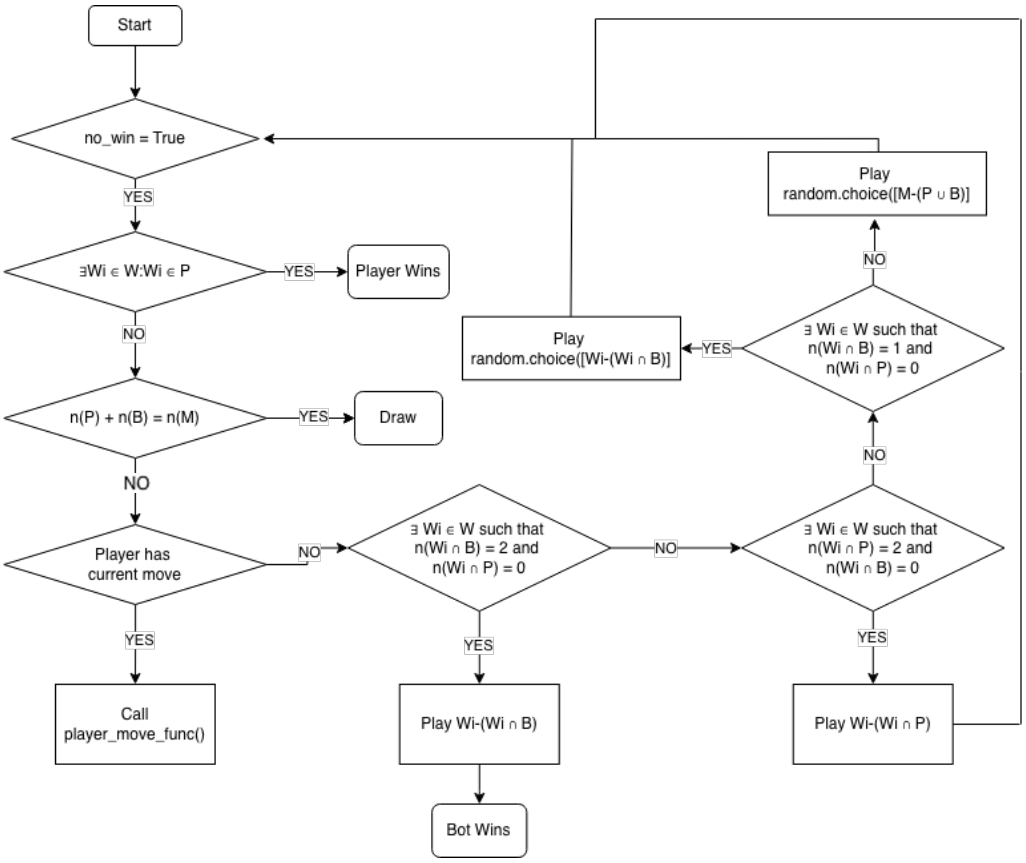
Case 1: Player-Bot-Player

Case 2: Bot-Player-Bot



The Main Loop

The algorithm kicks in after the 3rd move is played.



The Mathematical Side

Let P be the set of all player moves
Let B be the set of all bot moves
Let M be the set of all possible moves on the 3x3 grid
W contains all the possible winning combinations

$W = \{W_1, W_2, W_3, W_4, W_5, W_6, W_7, W_8\}$

In this situation, M is the universal set that contains P and B, i.e. $M = \mathbb{U}$

A few statements can be concluded:
$P \cap B = \emptyset$
$P \cup B \subseteq M$
$(P \cup B)' = \emptyset \iff P \cup B = M$
$P \subset M \rightarrow n(P) < n(M)$

Similarly, $B \subset M \rightarrow n(B) < n(M)$

Win condition

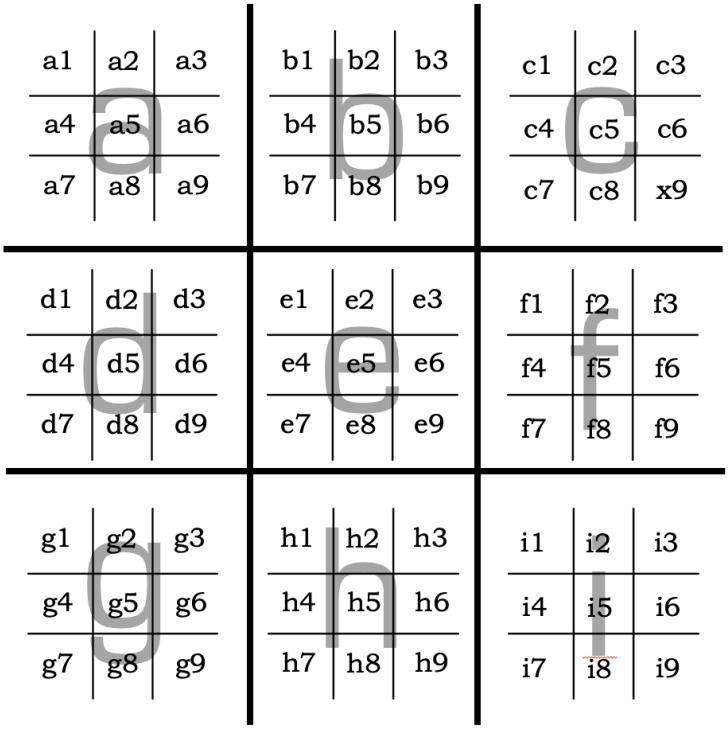
$\text{Win}(P) \iff \exists W_i \in W : W_i \subseteq P$
$\text{Win}(B) \iff \exists W_i \in W : W_i \subseteq B$

The Rules for Ultimate Tic Tac Toe

Ultimate Tic Tac Toe is played on a large 3x3 grid where each square contains its own smaller 3x3 Tic Tac Toe board. Players take turns placing Xs and Os in the small boards, but the position of your move determines where your opponent must play next. For example, if you place your mark in the top-right square of a small board, your opponent must make their next move in the top-right small board of the large grid.

If that destination board has already been won or is full, the opponent may play in any unfinished board. A player wins a small board by getting three in a row within it, and that board is then claimed by that player on the large grid. The overall objective is to win three small boards in a row (horizontally, vertically, or diagonally) on the large grid. If no player achieves this and all boards are completed, the winner is usually determined by who has won more small boards, though some rule sets declare a draw.

A key feature specific to UTTT is embedded in how the search transitions between depths: the cell played within a local board determines which local board the opponent is forced to play in next (cell 1 sends the opponent to board "A", cell 5 to board "E", and so on). If that target board is already decided, the next move again becomes a free choice across the whole board. This logic is recalculated at every node before the recursive call, ensuring the search tree correctly reflects the legal move structure of the game.



The Algorithm for Ultimate Tic Tac Toe

We made 2 algorithms for UTTT (Ultimate Tic Tac Toe).

Algorithm 1 – Heuristic [Suraj Gupta]

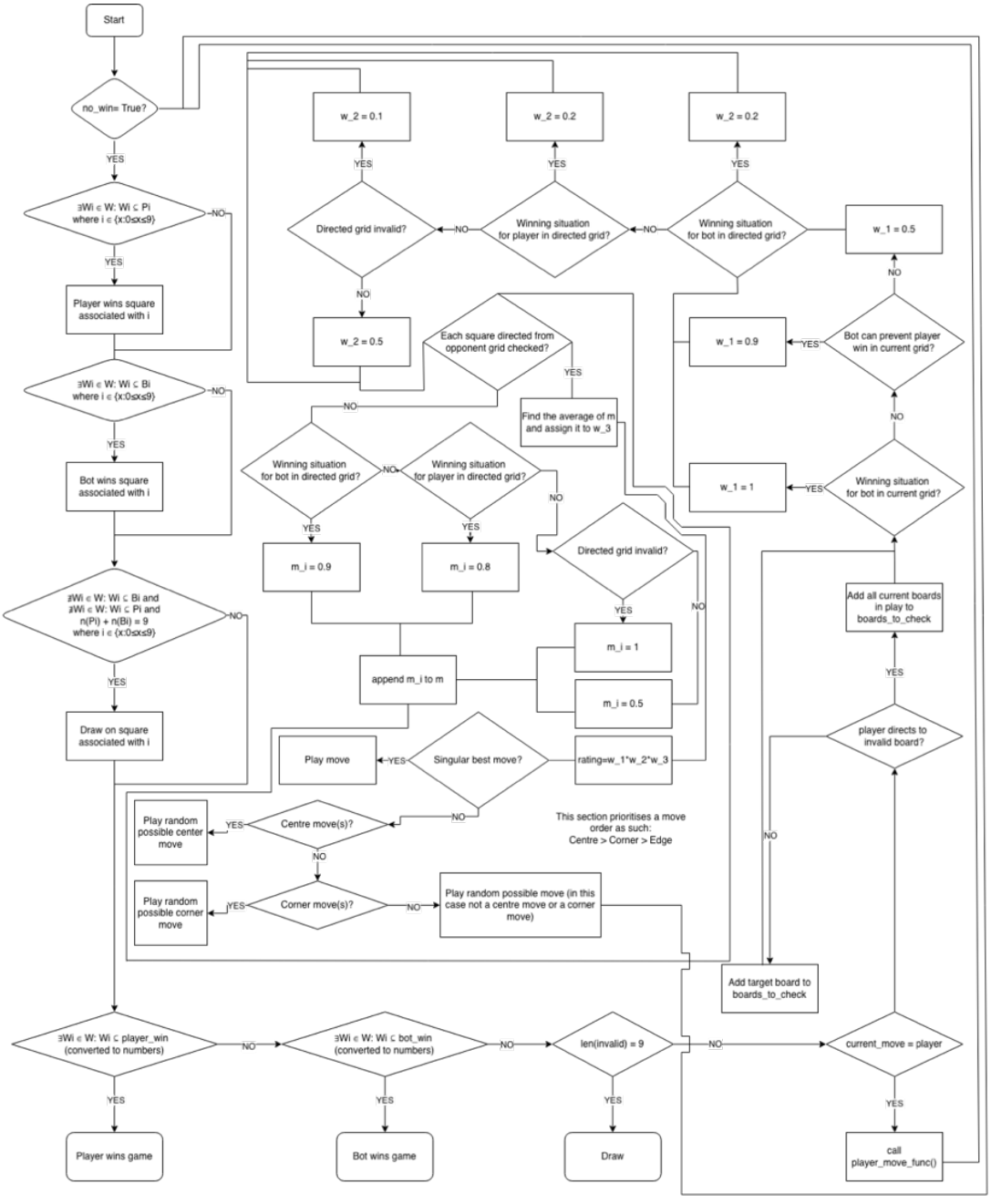
This algorithm uses an interesting approach. It assigns a rating to each grid on the current board. The rating takes 3 parameters- the merit of the move regarding the current board, regarding the directed board and regarding the possible further directed boards through a system which assigns a score to each move.

The First Three Moves

This algorithm determines the first three moves through a similar method in comparison to the original Tic Tac Toe algorithm. It begins with the center move and plays the center move in the next grid which it is directed to by the player's move. This algorithm treats the center move as the best move in general. Hence, even in the main algorithm, when there are multiple moves equally good, the algorithm assigns a priority to the center move.

The Main Algorithm

Given below is the flowchart of the algorithm used for determining the best move. It is a heuristic algorithm that assigns a rating to each move based on its merit. This rating system also takes into account future possibilities, however only upto 2-3 moves. It could be continuously improved by increasing the number of moves it can see into the future.



Optimising The Values Assigned to the Rating Inputs

w_1, w_2 and m_i (leading to w_3) are all assigned values based on the merit of the move. These values don't have any foundational basis other than the fact that they make sense. Playing a winning move in the current square is advantageous, but to what extent?

There's a way to find optimum rating values within a given range to assign to w_1, w_2 and m_i (hence w_3), which is through building an environment where bots assigned random ratings in the given ranges play against each other. The poor rating combinations are filtered out through elimination, and the bots assigned optimum rating combinations come out on top.

The code updates values.csv with the best rating combinations, which can be used in the actual algorithm. For the first test, 1000 bots each played 10 games with different rating combinations. The top 3 were (rounded to 3 decimal places):

```
w1_win,w1_block,w1_neutral,w2_win,w2_lose,w2_invalid,w2_neutral,w3_win,w3_lose,w3_inv
alid,w3_neutral
0.958, 0.765, 0.456, 0.710, 0.272, 0.103, 0.560, 0.729, 0.278, 0.200, 0.599
0.918, 0.779, 0.566, 0.820, 0.195, 0.048, 0.674, 0.905, 0.310, 0.176, 0.698
0.973, 0.772, 0.411, 0.918, 0.160, 0.132, 0.517, 0.767, 0.352, 0.193, 0.578
```

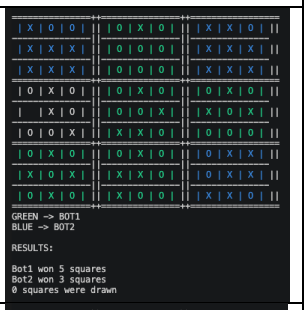
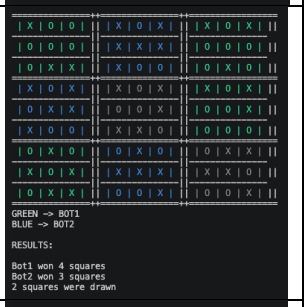
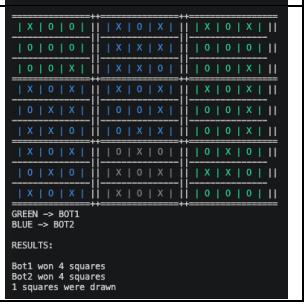
For the second test, 5000 bots each played 20 games with different rating combinations. The top 3 were (rounded to 3 decimal places):

```
w1_win,w1_block,w1_neutral,w2_win,w2_lose,w2_invalid,w2_neutral,w3_win,w3_lose,w3_inv
alid,w3_neutral
0.871, 0.804, 0.403, 0.803, 0.327, 0.084, 0.405, 0.896, 0.211, 0.184, 0.676
0.853, 0.819, 0.438, 0.833, 0.219, 0.045, 0.492, 0.741, 0.208, 0.101, 0.681
0.949, 0.878, 0.423, 0.890, 0.281, 0.149, 0.445, 0.891, 0.337, 0.185, 0.526
```

For the third test, 10000 bots each played 50 games with different rating combinations. The top 3 were (rounded to 3 decimal places):

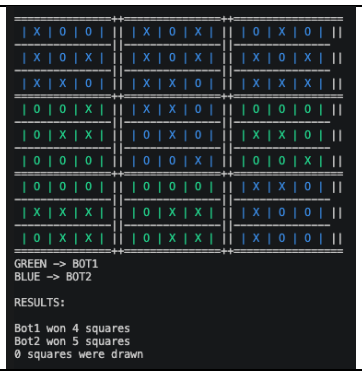
```
w1_win,w1_block,w1_neutral,w2_win,w2_lose,w2_invalid,w2_neutral,w3_win,w3_lose,w3_inv
alid,w3_neutral
0.966, 0.876, 0.477, 0.831, 0.344, 0.141, 0.562, 0.752, 0.370, 0.035, 0.655
0.915, 0.758, 0.469, 0.936, 0.334, 0.045, 0.693, 0.741, 0.295, 0.156, 0.600
0.972, 0.829, 0.562, 0.839, 0.267, 0.020, 0.444, 0.729, 0.375, 0.131, 0.638
```

Testing these values against the original values:

Tests	Bot 1 #Wins	Bot 2 #Wins	Result	Best Game
Test 1 (Rank 1)	Bot1 won 5 squares	Bot2 won 3 squares	Bot1 won	
	Bot1 won 4 squares	Bot2 won 4 squares	Draw	
	Bot1 won 4 squares	Bot2 won 4 squares	Bot1 won	
Test 2 (Rank 1)	Bot1 won 3 squares	Bot2 won 4 squares	Draw	
	Bot1 won 3 squares	Bot2 won 4 squares	Draw	
	Bot1 won 4 squares	Bot2 won 3 squares	Draw	
Test 3 (Rank 1)	Bot1 won 4 squares	Bot2 won 4 squares	Draw	
	Bot1 won 4 squares	Bot2 won 4 squares	Bot1 won	
	Bot1 won 4 squares	Bot2 won 4 squares	Bot1 won	

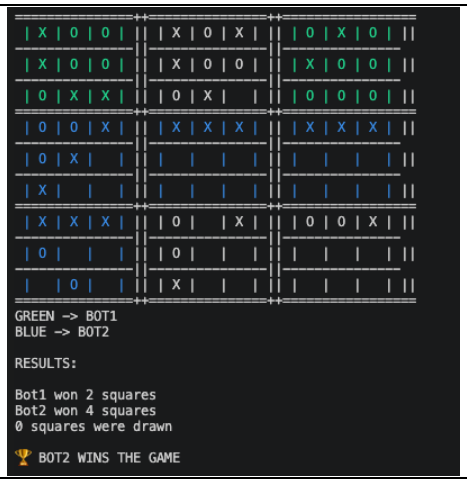
In each test, bot1 clearly performed better; however, interestingly, its performance declined in the second test before improving in the third.

Making Bot1 from Test 1 play against Bot1 from Test 3:

Bot 1 #Wins	**Bot 2 #Wins**	**Result**	**Best Game**
Bot1 won 3 squares	Bot2 won 3 squares	Draw	
Bot1 won 4 squares	Bot2 won 3 squares	Draw	
Bot1 won 4 squares	Bot2 won 5 squares	Bot2 wins	

Therefore, by a very small margin, Bot2 is better than Bot1, hence Test 3 proved to be effective, defeating the original bot and the bot from Test 1. It is possible to increase the number of moves the bot can evaluate in the future. It essentially would then go through all the possible corresponding moves of each possible move of the previous move.

It would take more time, but it would make the bot more efficient. We would technically be increasing the 'depth' of the bot. This script would be much more powerful than the previous one, even with the optimized values, which we can see in the tests. Bot1 is the original bot with the updated values and Bot2 is the new 5-ply bot

Result	Best Game
Bot2 wins	
Bot2 wins	
Bot1 wins	
Bot2 wins	
Bot2 wins	

Interestingly, the last and the second-to-last games were almost identical.

Algorithm 2 – Minimax [Nehul Malhotra]

The Algorithm

The Minimax algorithm is a decision-making algorithm commonly used in two-player zero-sum games such as Tic-Tac-Toe, Chess, and Checkers. It assumes that both players play optimally and attempts to find the move that maximizes the current player's chances of success. Thus the algorithm suggests the best possible outcome under optimal play by the opponent.

In UTTT, each state of the game (after a turn) can generate numerous legal moves (≤ 81 and ≥ 1), leading to a rapidly expanding search tree. The length of this tree is defined by a specified depth, which can be defined as the number of turns ahead into the future that the algorithm checks. At each depth, a specific state of the board can be seen affected by the move made in the previous depth of that branch. At each level, a move is decided, assuming the player is either “minimising” or “maximising”. Referring to the heuristic evaluation score determined by a function that assigns a numerical value to estimate the situation of the board, and hence the quality of this branch.

This property of “looking into the future” is an important feature of minimax, making it powerful, especially in a game such as UTTT, where the choice of the move not only affects the current board, but also the board that the opponent is to play in.

Alpha-Beta Pruning

The strength of Minimax, its exhaustive method leading to large branching, leads to a major challenge during its implementation. Even at relatively shallow depths, the number of positions to be evaluated grows exponentially.

To reduce computational cost, Alpha-Beta pruning was integrated; Alpha-Beta Pruning, essentially reduces branching by terminating or “pruning” a branch that won’t influence the final decision (move). This is achieved by maintaining two values; Alpha (α), the best evaluation score guaranteed so far for the maximising player, while Beta (β), the best evaluation score guaranteed for the minimising player.

Whenever a branch cannot improve the current best outcome (α or β), the branch is pruned. This helps to reduce branching without altering the final result.

Move Ordering and Pruning Efficiency

The effectiveness of Alpha-Beta Pruning is highly dependent on the order of the exploration of moves. If a move affecting the score strongly is found initially (having high α or β), pruning would occur earlier and more frequently.

Hence, to improve pruning efficiency, multiple move-ordering strategies were investigated. These strategies attempt to predict promising moves before an exhaustive full Minimax search and allow large portions of the search tree to be pruned.

We tested various strategies, such as a heuristic method to predict moves and even minimax at lower depths to help predict moves faster. This experimental testing showed that improved move ordering significantly reduced the number of evaluated positions without affecting the final decision.

Code Implementation

The algorithm described above was implemented via Python, with the board state represented as two dictionaries, nought_ and cross_, each keyed by the nine local boards (labelled “A” through “I”). Each key maps to a list of integers from 1-9, representing the cells that the player occupies within that local board. This representation avoids the overhead od copying a full board state at a node; instead, a move is simulated by simply appending a cell (1-9) to the relevant list, and undone afterwards, by popping it, keeping the search lightweight.

Win Detection

The function has_won() checks whether a set of played cells contains any of the eight winning cases (rows, columns and diagonals) by testing each line for a subset inclusion. This function is used at two levels: locally, to determine whether a player has won an individual board; and globally, to determine whether a player has won the overall game, by mapping the boards, “A” through “I” to their corresponding numeric positions, i.e. 1-9, and checking the same conditions.

Evaluation Function

Since an exhaustive search to the end of the game is computationally not feasible, the evaluate() function provides a heuristic estimate of the board’s situation or favourability towards a player, when the minimax reaches its maximum depth. Heuristic Weights are defined in a dictionary titled the same, the function rewards several features for each player, keyed in the dictionary, such as an unblocked two-in-a-row with a local board, occupying high value cells (centre > corner > edge), controlling a local board by holding a more cells in it as compared to the opponent, and most heavily — having won a local board. These boards, are also assessed as per their position, as are the cells.

```
WEIGHTS = {
    # overall board
    "center_win":    1000,
    "corner_win":    700,
    "edge_win":      500,

    "center_board":  50,     # Controlling center board
    "corner_board":  30,     # Controlling corner boards
    "edge_board":    20,     # Controlling edge boards
# local board
    "two_in_row":    10,     # Two in a row (local board)
    "center_moves":   5,     # Center cell (local board)
    "corner_moves":   3,     # Corner cell (local board)
    "edge_moves":     1
}
```

The final score is returned is the difference between the evaluating player's score and their opponent's, oriented so that a positive value always favours the player on whose behalf the evaluation is being done.

Move Ordering Implementation

The three move ordering strategies discussed earlier implemented within order_moves(), selected via the PRUNING_PRIORITY flag:

1. Under "nothing", moves are sorted using a fixed, static priority: centre cells first, then corners, and finally edges, without any evaluation of the position.
2. Under "heuristic", each move is scored individually: cell position is rewarded as before, and a large bonus is given if a cell would immediately complete a winning line on a local board, for either player, and is tested across both cross_ and noughts_. The heuristic addresses urgent calls, i.e. winning moves or a just a block, and doesn't consider other moves.
3. Under "minimax", the strategy instead draws on the result of a previous, shallower search (with a lower depth). A global variable, LAST_BEST_MOVE, stores the best move found at the previous iterative-deepening depth, and this move is placed first in the list if it remains legal. Since the best move at depth d is highly likely to remain strong at depth d+1, this ordering allows alpha-beta pruning to discard large portions of the tree very early in the search.

Minimax with Alpha-Beta Pruning

The minimax() function implements the recursive search itself. At each call, it first checks whether the game has already been won globally, if so returning a large terminal score (±10000); otherwise returning the heuristic evaluate score once the depth limit is reached. Legal moves are generated and passed through order_moves() before being explored, so that whichever ordering strategy is active can take effect.

Alpha and beta values are passed down through each call and updated as new moves are evaluated, and help in pruning as described earlier. An internal functioning flag additionally allows each trial move, its resulting score and any pruning event to be printed during the search, which proved useful for manually verifying correct pruning behaviour during development.

Move Selection and Iterative Deepening

Finally, get_best_move() serves as the entry point for choosing a move from a given board state. When PRUNING_PRIORITY is set to "minimax", the function performs iterative deepening: it runs a full search at depth 1, records the resulting best move in LAST_BEST_MOVE, then repeats the search at depth 2 with this information available to order_moves(), and so on up to the target depth. Each shallower pass therefore directly improves the move ordering — and consequently the pruning efficiency — of the next, deeper pass. Under the other two pruning priorities, the function instead performs a single search at the full target depth, without this incremental refinement.

Comparison

To compare pruning strategies, we ran 3 matches each with 2 games - the latter match having the player playing for cross, be playing for the nought instead, preventing variances due to the exhaustive computational time needed for the first move, having each cell open.

They were evaluated through a rating formula that considered multiple aspects.

$$\text{Rating} = \frac{C_G\left(S_w + \frac{S_D}{2}\right)}{T}, C_G = \begin{cases} 1 \text{ if win.} \\ 0.75 \text{ if draw} \\ 0.5 \text{ if loss.} \end{cases}$$

where C_G is the game coefficient, S_w is the no. of squares won, S_D is the number of squares drawn, and T is the average time per move

- In Match A, where Modified Minimax was tested against standard Minimax, both games resulted in draws. In the first game, Modified Minimax achieved a rating of **0.6150**, while standard Minimax scored **0.3922**. In the second game, standard Minimax scored **0.7007**, whereas Modified Minimax achieved **1.1231**.
- In Match B, Modified Minimax was tested against Heuristic Minimax. The first game was won by Modified Minimax, with ratings of **0.8685** for Modified Minimax and **2.2897** for Heuristic Minimax. The second game ended in a draw, with Heuristic Minimax scoring **0.6101** and Modified Minimax scoring **0.8475**.
- In Match C, standard Minimax was tested against Heuristic Minimax. Both games ended in draws. In the first game, standard Minimax obtained a rating of **0.4870**, while Heuristic Minimax scored **1.7787**. In the second game, Heuristic Minimax achieved **1.0057**, compared with **1.6791** for standard Minimax.

The ratings obtained by each algorithm across all four games were then averaged to produce an overall performance score. Standard Minimax achieved a final rating of **0.8147**, Modified Minimax achieved **0.8635**, and Heuristic Minimax achieved the highest overall rating of **1.4804**.

The Mathematical Side

Let P_i be the set of all player moves on board i where $i \in \{A, B, C, D, E, F, G, H, I\}$
Let B_i be the set of all bot moves on board i where $i \in \{A, B, C, D, E, F, G, H, I\}$
Let M_i be the set of all possible moves on board i where $i \in \{A, B, C, D, E, F, G, H, I\}$
In this situation, M_i is the universal set that contains P_i and B_i, i.e. $M_i = \mathbb{U}_i$

W contains all the possible winning combinations for the small board
$W = \{W_1, W_2, W_3, W_4, W_5, W_6, W_7, W_8\}$

W' contains all the possible winning combinations for the main board
$W' = \{W'_1, W'_2, W'_3, W'_4, W'_5, W'_6, W'_7, W'_8\}$

Let $B = \begin{bmatrix} S_A & S_B & S_C \\ S_D & S_E & S_F \\ S_G & S_H & S_I \end{bmatrix}$ where $S_i = \begin{cases} 1 \Leftrightarrow \exists W_i \in W : W_i \subseteq P_i \\ -1 \Leftrightarrow \exists W_i \in W : W_i \subseteq B_i \\ 0 \Leftrightarrow \nexists W_i \in W : (W_i \subseteq P_i \vee W_i \subseteq B_i) \text{ and } (P \cup B)' = \phi \\ \delta \text{ otherwise} \end{cases}$

Win condition
$\text{Win}(P) \Leftrightarrow \exists W'_i \in W' : \sum_{j \in W'_i} S_j = 3$

$\text{Win}(B) \Leftrightarrow \exists W'_i \in W' : \sum_{j \in W'_i} S_j = -3$

Draw condition
$\text{Draw} \Leftrightarrow \left(\forall W'_i \in W' : \left|\sum_{j \in W_i} S_j\right| \neq 3\right) \text{ and } (\forall S_i \in B : S_i \neq \delta)$

The Website – an Auxiliary Tool

To facilitate the development, testing, and evaluation of the algorithms, a dedicated web application was developed. It provides an implementation of Ultimate Tic Tac Toe, allowing users to play against the two algorithms or observe an automated match between the. It also serves as a visualisation tool, displaying legal moves, board states, and previous moves. Using the rating system previously developed, the website also calculates a rating of the players. The website is quite customizable with adjustable search depth and pruning strategy. The website perfectly sums up this project, providing a tool to understand the game and the algorithms better; https://ultimate-tictactoe.github.io/uttt/

Conclusion

This project demonstrates that both heuristic and Minimax-based approaches can be used effectively to play Ultimate Tic-Tac-Toe, each offering different advantages. While heuristic methods provide fast, practical decision-making, Minimax with Alpha-Beta pruning produces stronger strategic play by searching further into the game. Through testing, optimisation, and comparison, we showed how different algorithmic techniques influence performance and highlighted the balance between computational efficiency and playing strength in complex games.